1-D array ==> Data will be kept in 1 row and multiple columns

1) without new keyword
2) with new keyword

1-D arary without new ketword

```
int marks[] = {45, 99, 80, 76, 90, 100 };
```

1-D array with new keyword

```
int employeesAge[] = new int[5];
```

2-D array

==> If an array contains multiple rows and multiple columns, then it is 2-D array

==> 2-D array is a collection of many 1-D arrays

If we want toc reate a 2-D array, again we have 2 approaches
1) without new keyword
2) with new keyword

2-D array without new kewyord

```
char ticTacToeBoard[][] = {
        { 'X', 'O', 'X' },
        { 'O', 'X', 'O' },
        {'X', 'O', 'X'}
      };

      System.out.println(ticTacToeBoard[1][2]);
```

2-D array with new ketyword

```
    char tttBoard[][] = new char[3][3];
      tttBoard[0][0] = 'X';
      tttBoard[0][1] = 'O';
//       .....
```

Reading from 2-D array

```
char ticTacToeBoard[][] = {
    { 'X', 'O', 'X', 'O' },
    { 'O', 'X', 'O', 'X' },
    {'X', 'O', 'X', 'O'},
    { 'X', 'O', 'X', 'O' },
    { 'O', 'X', 'O', 'X' },
    {'X', 'O', 'X', 'O'}
      };

    for(int i = 0; i < ticTacToeBoard.length; i++) {
//        i = 0 ==> { 'X', 'O', 'X', 'O' }
      for(int j = 0; j < ticTacToeBoard[i].length; j++)
      {
//          System.out.println(ticTacToeBoard[i][j]);
        System.out.print(ticTacToeBoard[i][j]+" ");
      }
      System.out.println();
    }
```

Jagged Array

A Jagged array is 2-D array with rows having different no of columns!

```
//    marks of 3 students for 5 subjects

    int marks[][] ={
      { 10, 20, 30, 40, 50},
      { 11, 21, 31, 41, 51},
      { 12, 22, 32}
    };
```

3-D array

A 3-D array is a colelction of 2-D arrays!

==> Arrays is a built in class already existsed in java
==> This class contains various methods in it

1) toString() ==> To print the elements from a 1-D array

```
int marks[] = {45, 99, 80, 76, 90, 100 };
System.out.println(Arrays.toString(marks));
```

2) deepToString() ==> To print the elements from a 2-D array

```
int marks[][] ={
        { 10, 20, 30, 40, 50},
        { 11, 21, 31, 41, 51},
        { 12, 22, 32}
      };

    System.out.println(Arrays.deepToString(marks));
```

3) equals() ==> It checks whether 2 arrays are equal or not(both ararys length should be same and also elements should also be same)

```
    int arr1[] = {45, 99, 80, 76, 90, 100 };
    int arr2[] = {45, 99, 80, 76, 90 };

    System.out.println(Arrays.equals(arr1, arr2));
```

4) sort() ==> It is used to sort the array elements in either ascednign or descing order

```
//    int arr1[] = {45, 99, 80, 76, 90, 100 };
    Integer arr1[] = {45, 99, 80, 76, 90, 100 };

    System.out.println(Arrays.toString(arr1));

    Arrays.sort(arr1);//asc

    System.out.println(Arrays.toString(arr1));

    Arrays.sort(arr1, Collections.reverseOrder());//desc

    System.out.println(Arrays.toString(arr1));
```

5) copyOf() ==> This method is use to copy the elements from one array in to another array

```
    int arr1[] = {45, 99, 80, 76, 90, 200, 300 };

    int arr2[] = Arrays.copyOf(arr1, arr1.length);

    System.out.println(Arrays.toString(arr1));
    System.out.println(Arrays.toString(arr2));
```

6) fill() ==> It is used to fill every element in the arary with same value

```
int arr1[] = new int[5];

    Arrays.fill(arr1, 1);

    System.out.println(Arrays.toString(arr1));
```

Assigning one array to another array

when we assign one array to anotehr array, then java doesn't create that second new array, instead java make both array names point to a single existing array that got created

```
int a[] = {10, 20, 30};
    int b[] = a;

    System.out.println(Arrays.toString(a));
    System.out.println(Arrays.toString(b));

    a[0] = 100;

    System.out.println(Arrays.toString(a));
    System.out.println(Arrays.toString(b));
```

Vectors

Arrays and Vectors ==> 98% same

1) Arrays can be used to store mutliple values separted with commas
2) Vector can be used to store mutliple values separted with commas

```
Vector v = new Vector();

    v.add(10);
    v.add(20);
    v.add(30);

    System.out.println(v);
```

2) Arrays can store same type of data(homogeneous)
2) Vector can store both same and different types of data(Heterogenous)

3) Arrays size if fixed, it doesnt autoamtically increase or decrease the size
3) Vector size is intially 10, it means it can store 10 elements, but if we try to store extra elements, then vector automatically increases its size by ityself

4) Arrays doesnt support generics
4) Vector supports generics

```
Vector<Integer> v = new Vector();
```

<Integer> ==> GENERICS